Sonder — Presentation Topic Assignments

How to read this document

The product presentation (`sonder_presentation.pptx`) is 19 slides covering Sonder end-to-end. This document maps each presenter to the slides and subtopics they own. Owners should be prepared to go deep on their assigned material — the slides themselves are deliberately tight, so detail comes from the speaker.

Shreyas owns slide-flow, architecture overview, deployment, observability, the closing, and any cross-cutting topics not assigned to a single person.

Ali — Pinecone vector DB + LLM routing engine

Slides owned: 6 and 7.

Ali drives the conversation on how the system retrieves and how it routes language-model calls. Two interlocking pieces of intelligence: where the data lives in vector space, and which model gets asked what.

Slide 6 — Vector database design

Talking points:

- One Pinecone index, three namespaces (destinations, activities, cotravellers). Why one index over three: configuration centralisation, consistent dimension, easier cross-namespace coherence.
- 1536-dimensional embedding space, OpenAI `text-embedding-3-small`, unified across every surface. The same model produces vectors for cities, activities, traveller personas, and the 27 GoEmotions anchor glosses. Cross-namespace queries are coherent because everything lives in the same geometry.
- Embedding text shape — encoding the *feel* of a place rather than facts. 200-400 word context blocks per destination. For travellers: voice anchor + small-thing + quirks + persona-answer labels.
- Metadata-driven filters run before re-rank (budget feasibility, style, gender). Cosine retrieval surfaces the candidates; metadata filters cut the qualified pool; the feature pipeline ranks the survivors.
- Metadata-only updates as a schema-evolution pattern — the gender-backfill incident as a worked example.

Slide 7 — LLM stack + routing engine classifier

Talking points:

- Eight distinct models across small / large / validator / image / voice tiers. Each chosen for what it does best at lowest cost.
- The routing engine reads a per-tier provider preference. Anthropic primary, OpenAI fallback. Per-provider model identifiers so cross-provider failover never sends a Claude model id to OpenAI.
- Task-type classifier maps every call site to the right tier — `chat_reply` → small, `itinerary_generation` → large, `validate_itinerary` → validator, etc. Adding a new surface is one classifier entry.
- Why small-tier wins on persona voice: 25× cheaper, 3× faster, indistinguishable quality once the validator stack is layered on top.
- Streaming contract — the small-model output streams day-by-day so users see Day 1 of an itinerary within 15 seconds.

Mushahid — Backend layer, infrastructure, observability

Slides owned: 4, 12, 17, 18.

Mushahid drives the conversation on how the whole thing runs in production — the FastAPI application, the data stores, the real-time layer, the deployment topology, and the observability tooling. This is the operational story.

Slide 4 — Architecture overview (split with Shreyas)

Talking points specifically for Mushahid:

- The FastAPI process — REST routes, server-sent-event streams, WebSocket endpoints, and the synthetic-agents background loop all in one lifespan.
- Firestore (named-database mode) holds operational state: user profiles, generated itineraries, chat sessions with messages as a subcollection, shared-itinerary negotiation logs, social posts.
- Firebase Authentication for identity, Firebase Storage for binary assets (synthetic-persona avatars, cached voice MP3s).

Slide 12 — Real-time layer

Talking points:

- Two WebSocket endpoints: `/ws/chat/{session_id}` for live conversations, `/ws/notifications` for the global per-user channel.
- First-message authentication — tokens go in the first JSON payload after handshake, never in query strings. Why this matters: query-string tokens leak through browser history, server logs, referer headers.

- TTL-derived presence — clients ping every 30 seconds, presence is `(now – last_seen) < 90s` rather than a boolean `online` flag. Booleans go stale on dropped connections.
- Firestore real-time listeners for the shared-itinerary surface — Firestore guarantees ordering and delivery that would have required significant engineering on top of raw WebSockets.
- Modular separation across four team-owned folders with strict import boundaries.

Slide 17 — Observability

Talking points:

- Sentry for errors. The deliberate noise filter: drop `Failed to fetch` TypeErrors (client lost internet — not a bug) and expected 4xx (404 on first profile fetch, 401 during auth transitions). Only 5xx and genuine uncaught exceptions reach the dashboard.
- PostHog for product analytics. Five top-level metric families: user satisfaction, retrieval quality, response quality (validator telemetry), hallucination rate, completion funnel.
- Why two tools rather than one: Sentry's UI is built for incident triage, PostHog's for cohort analysis. Conflating them in one platform would compromise both.
- The continuous semantic-genericity score as a leading drift indicator — visible weeks before any individual reply looks broken.

Slide 18 — Deployment

Talking points:

- Cloudflare Pages — global edge network, 300+ points of presence. Catch-all Pages Function proxies `/api/*` to Render so the SPA stays on a single domain (required for the web-push service worker registration).
- Render — long-running FastAPI container, auto-deploys from `main`, TLS termination, health-check loop. Designed to scale to multiple replicas with one config change (Redis pub/sub for the WebSocket connection manager).
- Multi-provider language model layer — operational independence means partner outages don't cascade.
- The actual meaning of "designed at scale" — not running at scale today, but being ready to without re-architecting.

Jahnvi — User persona experience, ranking, learning algorithms

Slides owned: 5, 9, 11, 15.

Jahnvi drives the conversation on how Sonder learns about a user, surfaces matches that feel calibrated rather than uniformly hyped, and self-corrects when feedback comes in. This is the personalisation story.

Slide 5 — Foundation datasets and frameworks

Talking points:

- Push-Pull Motivation Theory (Dann 1977, Crompton 1979) — the academic framework grounding our 12-dimension persona space. Six push dimensions (escape, novelty, connection, reflection, curiosity, prestige) and six pull dimensions (nature, culture, food, nightlife, comfort, exploration).
- Substring-matched keyword sets — why this matters for *auditability*. When the system tells a user they score high on `escape_reset`, we can point at the exact phrases that contributed.
- GoEmotions (Demszky et al. 2020) — 27 emotion labels used as anchor vectors in our embedding space. We don't ship the 58k training rows; we embed the 27 tone-anchored glosses once and classify by cosine similarity.
- The eight-key emotional-signature taxonomy — synthesised specifically for travel personas. Private framing for every persona-voiced surface; key itself never shown to the user.
- Three lenses at three granularities: fine emotion → motivational structure → voice signature.

Slide 9 — Co-traveller matching

Talking points:

- The ranker as a stage-based engine that runs the same code path across three matching surfaces (co-traveller, destination, activity), with each surface declaring its own policy.
- Ten reusable scoring functions in the registry. Equal-weight priors across features — one over N — because we haven't yet earned data-grounded confidence in per-feature importance.
- Hard pre-ranking filters fire *before* scoring: budget feasibility, avoid-list veto, travel-style hard filter, same-gender filter for solo travellers.
- The calibration story — score-as-probability rather than score-as-similarity. A 0.6 honestly means roughly a 60% chance of mutual approval. This makes denial *meaningful*. The pure-cosine alternative produces uniformly high approval rates that hide compatibility signal.

Slide 11 — Learning loop (the self-correction story)

Talking points:

- User feedback applies *weight penalties* on the ranker's cost function. Saying "cheaper" maps to budget-fit features. "Less packed" maps to pace features. "More local" maps to interest-overlap features.
- Decay across turns — turn 1 full strength, turn 2 half, turn 3 quarter, floor at 1/8. Prevents oscillation when a user keeps pushing back on similar things.

- The self-correction loop is not a record-keeping update. After weights are written, the **same generation pipeline re-runs end-to-end** — retrieval, ranking, generation, validation — with the new weights applied. The system literally re-prices its own previous recommendation.
- Live chat signal scanner — sarcasm-aware (blocks boosts on `/s`, "said no one ever", eye-roll emoji), negation-aware (negation zone extends five words or to the next clause break, so "I don't love crowded places" doesn't boost crowded-tag interest).
- Auditable end-to-end. Every revision turn records the feedback, scope, target days, dropped + added titles, which feature weights moved by how much, and the validator's verdict.

Slide 15 — Cinematic destination reveal

Talking points:

- The approve-flow UX as the emotional peak of the product. The moment a user locks in their trip deserves a beat.
- Ken-Burns photo montage — five destination images cycle every 1.2 seconds with a slow zoom.
- Typographic build — *LOCKED IN* tracks out, *COUNTRY* drops in, *CITY* slams from 1.55× scale + 16px blur to 1×.
- Gold-dust particles under a cinematic vignette. Trailer cadence, not browser tab.
- Co-traveller prompt after the reveal: "Want to find someone to travel with?" Yes routes to the matching intake, no to dashboard.

Shreyas — Architecture solution design, deployment overview, closing

Slides owned: 1, 2, 3, 4, 8, 10, 13, 14, 16, 19.

Shreyas owns the framing — what problem we set out to solve, what we built that nobody else does, how the system is designed at scale, and how the team thinks about the harder parts (validators, shared-itinerary negotiation, synthetic populations). This is the architectural narrative.

Slide 1 — Title

Open the deck. Sonder, the tagline, one breath.

Slide 2 — The problem

Talking points:

- Walk through each existing platform — TripAdvisor, Google Travel, Instagram, Airbnb — and name what each one specifically fails at.

- The harder unsolved problem: *who you travel with*. Solo travellers get nothing. Couples get hotel suggestions. Anyone wanting to share a trip with a compatible stranger falls back to hostel bars and dating apps.

Slide 3 — What Sonder is

Talking points:

- Three things that nobody else does together: real-venue itineraries, calibrated compatibility matching, and a believable cold-start social layer through synthetic travellers.
- The intent: feel like a friend's recommendation, not a search engine result page.

Slide 4 — Architecture overview (intro + split with Mushahid)

Talking points specifically for Shreyas:

- Open with the four service planes as a framing diagram. Mushahid takes the deep dive on the backend, Firestore, and infrastructure.
- The Cloudflare-edge / Render-backend / Pinecone-vectors / Firestore-state separation as the architectural philosophy — operational independence at scale.

Slide 8 — RAG itinerary generation (architecture)

Talking points:

- The end-to-end pipeline: three parallel persona-inference pipelines → destination retrieval → activity retrieval → ranking → grounded generation → validator critic → optional refinement loop.
- Information starvation as quality control — the architectural decision that shows up across the system. The persona-inferring LLM never sees the embedding vector or the GoEmotions output. The validator never sees the user prompt. The matcher's policy file never sees individual user data.
- Streaming JSON parser yields each day the moment its closing brace lands. UX consequence: user is reading Day 1 within 15 seconds while Day 7 is still being written.

Slide 10 — Validator engine

Talking points:

- Five surface-specific critic prompts. The chat-reply validator is the most active, watching ten categories (assistant voice, AI leakage, semantic drift, token stutter, romantic vibes, taxonomy leakage, unsafe content, bad conversation dynamics, contradiction against established chat memory, plus repair).
- Deterministic local pre-check runs first — regex-based, instant, no LLM cost. Kills cheap failures pre-LLM and reserves model spend for the genuinely ambiguous cases.
- Three-block trip-scope guardrail in every persona prompt: hard trip scope, PPM as private framing, emotional signature as private framing. The reason a Paris persona never opens with "I see you're

interested in Lisbon too?" on a Japan trip.

- Async edit-in-place repair — reply broadcasts immediately, validator runs asynchronously, repaired text swaps in via a `message_edited` WebSocket event. Large-model-validated quality at small-model latency.

Slide 13 — Shared-itinerary negotiation

Talking points:

- Both sides edit one document mediated by the proposal-counter-accept loop. Direct edits would race; mediated edits are explicit, reviewable, and persisted.
- No hard reject state — the persona evaluator must either accept or counter, never stonewall. The negotiation always moves forward.
- Reason-in-message rule — every counterproposal carries a brief conversational reason like "hakone feels far after the museum day".
- Optimistic locking via version field. 409 on mismatch, client re-fetches and retries — never silent overwrites.
- Token-Jaccard dedupe prevents the persona from circulating the same idea twice. If the LLM tries, the verdict flips to accept rather than looping.

Slide 14 — Synthetic travellers (the cold-start solution)

Talking points:

- 192 LLM-designed personas + 18 couples seeded into the matching pool because a travel app with no users feels dead.
- Two-stage blind-writer pipeline. Stage 1 LLM receives only city, age, gender, and four persona-question option keys. It writes the character. Stage 2 inferrer assigns PPM dimensions and emotional signature blind to the writer's output.
- *A language model with the answer key in front of it writes to that key. A language model that has to guess writes something honest.*
- Diversity matrix is locked: 16 cities × 3 age buckets × 2 genders × 2 per cell. Backfill scripts patch metadata without re-paying LLM cost — pattern reused for every future schema-only field addition.

Slide 16 — Group-style filtering

Talking points:

- Four user types as four genuinely different products. Trying to make one ranker serve all four would compromise every one.
- Per-group itinerary planning rules — solo gets counter-seating venues and walking-distance stops, couples get every-overnight-private, families get kids-friendly menus and dinner-by-7pm, friends get

split-and-rejoin activities and apartment-over-rooms.

- Why family and friends matching is *disabled* — they already have their travelling party. Surfacing strangers as "companions" makes no product sense.

Slide 19 — Closing

Land the talk. *"Travel, together."* Repo link. Take questions.

Speaker order

1. **Shreyas** — slides 1, 2, 3, 4 (intro half), then hands to Mushahid mid-slide-4. 2. **Mushahid** — slide 4 (backend half). 3. **Jahnvi** — slide 5. 4. **Ali** — slides 6 and 7. 5. **Shreyas** — slides 8. 6. **Jahnvi** — slide 9. 7. **Shreyas** — slide 10. 8. **Jahnvi** — slide 11. 9. **Mushahid** — slide 12. 10. **Shreyas** — slides 13, 14. 11. **Jahnvi** — slide 15. 12. **Shreyas** — slide 16. 13. **Mushahid** — slides 17, 18. 14. **Shreyas** — slide 19.

Format and delivery notes

- Total runtime target: ~25-30 minutes for the slide content, 10-15 minutes Q&A.
- Each slide is intentionally tight on text. Detail comes from the speaker, not the slide.
- Live demo (if time allows): the cinematic reveal on `/trip-locked-in`, the matching surface with the same-gender filter, and a synthetic-persona chat to show the persona voice + edit-in-place validation in action.
- Backup material: the README.md (`c:\root\sonder-ali-fix\README.md`) and the full project report (`reports/sonder_project_report.pdf`) are the source of truth for any detail not on a slide.